

**Zyter
X
CMU Knowledge Graph and Multi-Agent Voice
Orchestration System Report**

Team Members- Lohitaksha Ananthula, Jingjie Shao, Sijia Ma, Xiaoyang Ma,
Xiaoyu Song
Faculty Advisor- Anand Rao

Table of Contents

<u>1. EXECUTIVE SUMMARY.....</u>	<u>4</u>
2. INTRODUCTION.....	5
2.1. BACKGROUND.....	5
2.2. PROBLEM STATEMENT	5
2.3. PROBLEM OBJECTIVES.....	6
2.4. LITERATURE REVIEW AND RELATED WORK.....	6
2.5. BUSINESS IMPACT	7
2.5.1. OPERATIONAL EFFICIENCY & POLICY AUTOMATION	7
2.5.2. CLINICAL OUTCOMES & COST AVOIDANCE.....	7
<u>3. DOCUMENTATION.....</u>	<u>8</u>
<u>4. KNOWLEDGE GRAPH (POLICY AUTOMATION SYSTEM)</u>	<u>8</u>
4.1. SCOPE OF WORK	8
4.2. SYSTEM ARCHITECTURE.....	8
4.3. POLICY PROCESSING ARCHITECTURE	10
4.4. PATIENT RECORD PROCESSING ARCHITECTURE	12
4.5. COMPLIANCE EVALUATION ARCHITECTURE.....	13
4.6. VALIDATION	14
4.6.1. METHODOLOGY: STRUCTURED LLM-AS-A-JUDGE.....	14
4.6.2. PHASE 1: DATA DICTIONARY EXTRACTION	14
4.6.3. PHASE 2: POLICY CONDITION EXTRACTION	15
4.6.4. PHASE 3: PATIENT CONDITION EXTRACTION	16
4.6.5. JOIN TECHNICAL VALIDATION	18
4.6.6. FUTURE CLINICAL VALIDATION.....	18
4.7 LIMITATIONS AND FUTURE WORK	18
<u>5. VOICE AGENT (POST-DISCHARGE ORCHESTRATION SYSTEM)</u>	<u>19</u>

5.1. SYSTEM ARCHITECTURE AND TECHNOLOGY STACK	19
5.1.1. OPERATIONAL CHALLENGES IN POST-DISCHARGE CARE	19
5.1.2. MODULAR MULTI-AGENT ARCHITECTURE	20
5.1.3. KEY TECHNICAL COMPONENTS.....	21
5.2. CORE AGENT LOGIC AND IMPLEMENTATION.....	22
5.3. METHODOLOGY AND DEVELOPMENT APPROACH.....	23
5.3.1. CLINICAL WORKFLOW ANALYSIS AND THREAT MODELING	23
5.3.2. PROMPT ENGINEERING STRATEGY	23
5.3.3. DESIGN PRINCIPLES: DUAL-LAYER SAFETY	23
5.3.4. ITERATIVE VALIDATION STRATEGY	24
5.3.5. LOGGING AND AUDITABILITY.....	24
5.4. VALIDATION AND DATA ANALYSIS	24
5.4.1. DATA SOURCES AND LIMITATIONS	24
5.4.2. VALIDATION OUTCOMES	25
5.4.3. CONFUSION MATRIX ANALYSIS	25
5.5. FUTURE WORK.....	26
 <u>6. LESSONS LEARNED</u>	 <u>27</u>
 <u>7. CONCLUSION</u>	 <u>28</u>
 <u>8. RECOMMENDATIONS.....</u>	 <u>28</u>
 <u>REFERENCES.....</u>	 <u>30</u>
 <u>APPENDIX.....</u>	 <u>32</u>
 APPENDIX A: TECHNICAL IMPLEMENTATION AND DEPLOYMENT	 32
APPENDIX B: DETAILED VALIDATION METRICS BY MODEL & STRATEGY.....	35

1. Executive Summary

Modern healthcare delivery is currently burdened by two critical fragmentation points: the manual, inconsistent interpretation of complex medical coverage policies and the lack of structured support for patients after hospital discharge. These gaps result in significant administrative overhead for providers and contribute to high preventable hospital readmission rates. Zyter TruCare addresses these challenges through its integrated digital health platform, and this Capstone project was commissioned to enhance that platform with advanced AI capabilities. The team was tasked with engineering a scalable solution to bridge the gap between back-office payer operations and front-end patient care, with the specific goal of eliminating manual policy review bottlenecks and providing continuous, safety-critical support to patients post-discharge.

To address these objectives, the team developed the Zyter-CMU Knowledge Graph and Multi-Agent Voice Orchestration System. The development process employed a hybrid architecture that combines the reasoning flexibility of probabilistic Large Language Models (LLMs) with the strict safety of deterministic Knowledge Graphs. Using tools such as Python, LangGraph for orchestration, and SQL for structured querying, the team engineered a dual-component solution. The first component, Policy Automation, automatically translates unstructured PDF medical policies into computable SQL rules and visual Knowledge Graphs. The second component, Patient Voice Orchestration, is a multi-agent voice assistant capable of providing 24/7 symptom triage, medication education, and appointment management.

The solution demonstrated high reliability in validation testing, delivering measurable operational and clinical impact for the client. The Policy Automation component achieved a 0.90 F1-score in data dictionary extraction and successfully generated executable SQL queries for complex policies, effectively transforming a manual review process into an automated pipeline. This allows Zyter to reduce claim denials by visually pinpointing missing requirements prior to submission. Regarding patient safety, the Voice Orchestration system achieved 96.8% sensitivity in detecting emergency symptoms and maintained 100% accuracy in intent routing with an average latency of less than 1.47 seconds. By integrating these tools, Zyter can gain a competitive advantage that streamlines administrative workflows while simultaneously working to reduce hospital readmissions by identifying potential complications early.

2. Introduction

2.1. Background

Hospitals and healthcare organizations face increasing pressure to ensure continuity of care after a patient leaves the hospital. Many patients experience confusion about follow-up appointments, symptom monitoring, or medication instructions, contributing to a 19.6% 30-day rehospitalization rate and an estimated \$17.4 billion in unplanned costs annually (Jencks et al., 2009). At the same time, clinical teams must interpret a growing number of complex medical policies and coverage guidelines, often written in dense natural language and applied manually. This reliance on manual processes represents a significant workflow gap, as automatic provision of decision support is widely cited as the strongest independent predictor of improved clinical practice (Kawamoto et al., 2005). These processes can be time-consuming, inconsistent, and difficult to audit, especially when multiple departments or care providers are involved. As healthcare demands grow, there is a need for systems that can support both reliable, automated policy interpretation and structured patient engagement throughout the post-discharge period.

2.2. Problem Statement

The project addresses two categories of challenges that weaken care continuity. First, medical and operational policies are complex, frequently updated, and interpreted differently across teams, making it difficult to ensure consistent decisions and clear justification. Manual interpretation limits scalability, introduces delays, and lacks a transparent audit trail for why a specific coverage decision was made.

Second, patients leaving the hospital often lack support in managing day-to-day tasks such as identifying concerning symptoms, understanding medication instructions, or navigating appointments. Without timely guidance, patients face uncertainty, which can increase the likelihood of adverse events. A unified, structured system is needed to improve both decision reliability for staff and communication quality for patients.

2.3. Problem Objectives

The project aims to design and integrate two complementary components. The first component is a Knowledge Graph-based policy automation system that utilizes Multi-Agent Large Language Models (LLMs) to convert unstructured policy text into computable SQL rules and graph representations. Its objective is to enhance consistency, transparency, and efficiency in policy-based decision-making by visually mapping clinical data against coverage requirements.

The second component is a multi-agent voice orchestration system that supports patients after discharge. By enabling appointment management, symptom reporting, medication questions, and caregiver communication, the system provides structured, safe, and policy-aligned guidance throughout recovery. Together, these components establish a foundation for scalable, AI-assisted clinical support.

2.4. Literature Review and Related Work

The case for automating post-discharge care is driven by significant gaps in current healthcare transitions identified in major clinical studies. Jencks et al. (2009) established the foundational evidence for this crisis, identifying a direct link between readmissions and fragmented care, noting that approximately half of readmitted patients lacked any outpatient follow-up prior to their return. While clinical teams attempt to manage this risk, they are hindered by manual policy interpretation. A study in the Journal of the American Medical Informatics Association argues that medical knowledge must transition from “dense natural language” to computable frameworks (Boxwala et al., 2011), a shift supported by a British Medical Journal systematic review that identified automatic provision of decision support as the strongest predictor of improved clinical outcomes (Kawamoto et al., 2005).

To bridge the gap between complex policies and patient engagement, this project leverages Knowledge Graphs and Conversational AI. While a systematic review in JAMIA confirmed the efficacy of conversational agents for chronic disease management (Laranjo et al., 2018), an audit study in the BMJ warned of historical inaccuracies in digital triage tools (Semigran et al., 2015). Therefore, the system prioritizes deterministic safety. By grounding generative AI in structured Knowledge Graphs (Rotmensch et al., 2017) and enforcing the specific risk management and

human oversight principles mandated by the WHO and NIST (WHO, 2021; NIST, 2023), the system targets the high-performance medicine model envisioned by Topol (2019).

2.5. Business Impact

The combined system offers measurable return on investment (ROI) for healthcare organizations by addressing the twin challenges of administrative inefficiency and preventable readmissions.

2.5.1. Operational Efficiency & Policy Automation

The Knowledge Graph architecture targets the high cost of manual administrative workflows. Current manual interpretation of complex medical policies creates significant bottlenecks, which this system mitigates through automation:

- **Reduced Administrative Burden:** Industry analysis indicates that AI-driven policy automation can reduce administrative turnaround times by up to 50% (Davenport & Kalakota, 2019). By automating routine eligibility checks, staff can reallocate significant hours from paperwork to patient-centered activities.
- **Immediate Cost Avoidance:** The administrative cost to rework a single denied claim is estimated between \$25 and \$118 depending on complexity (Change Healthcare, 2022). By standardizing eligibility decisions upstream, the system prevents these errors before submission, directly eliminating these rework costs for provider networks.

2.5.2. Clinical Outcomes & Cost Avoidance

The Voice Agent's proactive monitoring capabilities directly impact the financial bottom line by identifying complications early and preventing costly hospital returns:

- **Financial Retention via Readmission Reduction:** Evidence from safety-net health systems demonstrates that AI-enabled post-discharge monitoring can significantly reduce readmission rates. One notable implementation saw 30-day readmission rates drop from 27.9% to 23.9%, enabling the health system to retain \$7.2 million in at-risk pay-for-performance funding (AJMC, 2025).
- **Patient Engagement ROI:** Digital engagement platforms have been shown to drive a 29% reduction in overall hospital readmissions for engaged patient cohorts compared to non-engaged control groups (PMC, 2024). This validates the financial impact of the

continuous, automated support provided by the voice orchestration agents compared to standard care.

3. Documentation

A comprehensive technical guide is provided to support the deployment and maintenance of both the Medical Policy Knowledge Graph and the VoiceAgents system. This documentation covers the complete software architecture, including the multi-agent Python pipelines, LangGraph orchestration, and API configuration (Gemini, Anthropic, OpenAI). It also includes detailed repository structures, dependency management, and step-by-step instructions for launching the Streamlit interfaces and command-line tools.

For full implementation details, codebase structure, and execution commands, please refer to **Appendix A**.

4. Knowledge Graph (Policy Automation System)

4.1. Scope of Work

The primary objective of this project is to develop a comprehensive computational system capable of translating complex medical policy documents into computable rules that enable automated evaluation of patient eligibility for medical procedures. This objective addresses a critical gap in healthcare operations where policy interpretation remains largely manual, time-consuming, and subject to inconsistent application across different reviewers and organizations.

4.2. System Architecture

This section describes the high-level architecture of the Knowledge Graph Policy Automation System. As illustrated in **Figure 1**, the system is built upon a dual-stream pipeline that processes medical policies and patient records in parallel before converging at a decision engine. This architectural approach enables a clear separation of concerns, allowing for independent optimization of policy logic extraction and clinical data parsing.

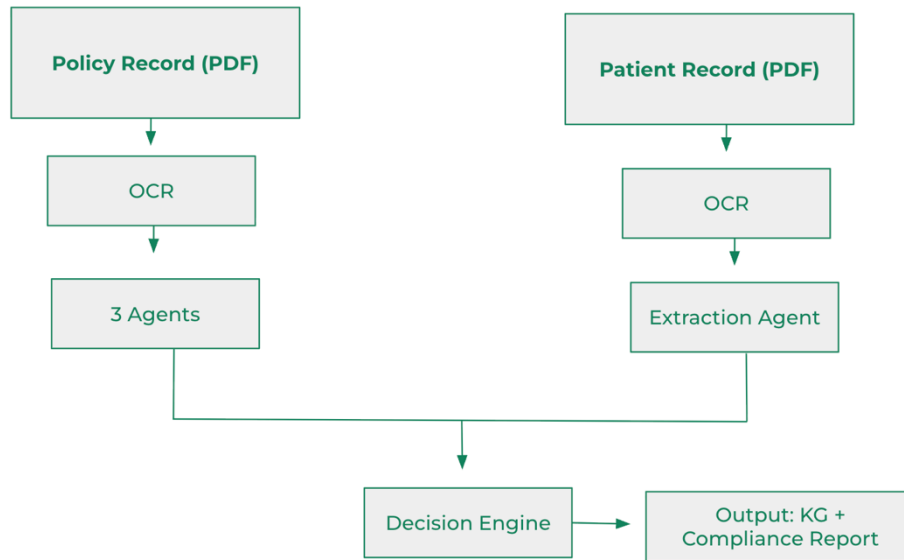


Figure 1. High-Level System Architecture showing parallel processing streams

As shown in **Figure 1**, the architecture consists of three distinct phases:

- **Policy Processing Stream (Left Branch):** Accepts medical policy documents in PDF format and transforms them through a multi-agent pipeline into structured, computable rules.
- **Patient Record Processing Stream (Right Branch):** Accepts unstructured patient medical records (PDF) and transforms them into structured data using a specialized extraction agent. This phase must handle the considerable variability in medical documentation formats and terminology. For instance, a single diagnosis like "Hypertension" may appear as "HTN" in a physician's note, "I10" in an ICD code list, or "Elevated BP" in a vitals chart. Handling this variability is a challenging task for clinicians and automated systems alike, requiring context-aware parsing rather than simple keyword matching.
- **Decision Engine (Convergence):** Brings together the outputs of the first two phases to systematically evaluate eligibility. This phase generates the final Compliance Report and Knowledge Graph (KG) visualizations, which allow for rapid human verification of the automated decision.

4.3. Policy Processing Architecture

The first stream, Policy Processing, is responsible for transforming narrative text into executable logic. This transformation is not a simple extraction task but rather a sophisticated interpretation process that must understand medical terminology, recognize logical structures, and formalize these elements into computable expressions.

Example of Complexity: Consider the Bariatric Surgery Policy (CG-SURG-83) used in this project. The eligibility criteria are not linear; they contain nested dependencies. The agent must interpret that a BMI ≥ 35 is only qualifying if a comorbidity (like Diabetes) is also present, whereas a BMI ≥ 40 stands alone. Capturing this logic requires parsing the text into a structured Boolean expression— $((\text{BMI} \geq 40) \text{ OR } (\text{BMI} \geq 35 \text{ AND Comorbidity} = \text{True}))$ —a task that standard keyword extraction cannot perform reliably.

This process is orchestrated by a sequential multi-agent pipeline, detailed in **Figure 2**.

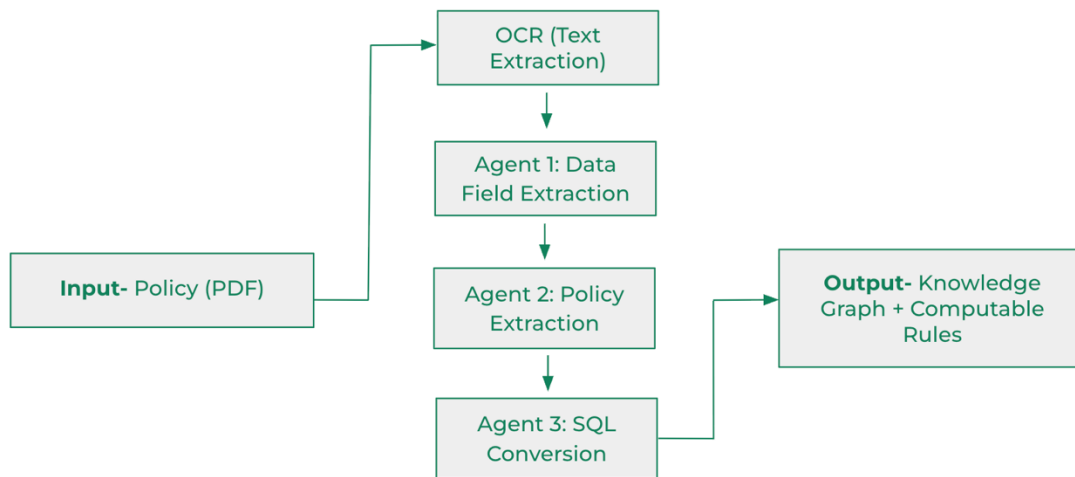


Figure 2. Detailed Policy Extraction Architecture (Multi-Agent Pipeline)

The pipeline progresses through four specific stages:

1. **OCR (Text Extraction):** The process begins with OCR to convert PDF-based medical policy documents into machine-readable text. Because policies often contain complex layouts such as multi-column text and tables, the system adopts a dual-technology approach using PyMuPDF and Tesseract. This design addresses common healthcare OCR challenges, such as table cell reconstruction and reading order preservation in dense clinical documents.
2. **Agent 1: Data Field Extraction:** Agent 1 is responsible for constructing the Policy Data Dictionary. Using the OCR-extracted text, it identifies all data fields referenced or implied by the policy.
 - Example: For the Bariatric policy, the agent identifies and types the fields: patient_bmi (Float), patient_age (Integer), and weight_loss_program_history (Boolean).
3. **Agent 2: Policy Extraction:** Agent 2 focuses on Eligibility Logic Extraction. It takes the narrative policy text and decomposes complex statements into atomic logical rules linked to the fields defined by Agent 1.
 - Example: The agent extracts the "Medical Necessity" section and outputs the structured logic JSON:

```
{  
  "condition": "BMI 40 or greater OR BMI 35 with  
comorbidity",  
  "logic": "(patient_bmi >= 40.0 OR (patient_bmi >=  
35.0 AND comorbidity_flag = TRUE))"  
}
```

4. **Agent 3: SQL Conversion:** The final agent translates the structured policy rules into an executable SQL WHERE clause. It handles comparison operators and applies logical grouping with correct parenthesization.

- Example: The agent converts the JSON logic above into the executable query: WHERE patient_age >= 18 AND (patient_bmi >= 40.0 OR (patient_bmi >= 35.0 AND comorbidity_flag = TRUE))

4.4. Patient Record Processing Architecture

The second stream, depicted in the **Right Branch of Figure 1**, addresses the challenge of transforming unstructured patient medical records into structured data.

- **OCR for Patient Medical Records:** Patient records are processed using the same dual-technology OCR pipeline. This stage is robustly designed to handle multi-document clinical files and varying image quality caused by scanning or faxing, which often introduces noise that can obscure numerical values like lab results.
- **Extraction Agent (Medical Record Parser):** Guided by the Data Dictionary produced in the policy phase, the Extraction Agent interprets clinical context to populate the required fields.
 - Example: When processing **Patient 8472202544**, the agent scans the "History of Present Illness" section to extract patient_bmi: 42.4 and sets comorbidity_flag: true based on a diagnosis of Type 2 Diabetes found in the "Assessment" section.

Output: KG + Compliance Report. In the final step, the system generates a **Patient Knowledge Graph**. As shown in **Figure 3**, the graph visually maps the patient's specific data against the policy's logical structure. Green nodes indicate satisfied conditions (e.g., "Age requirement" and "BMI 40 or greater"), while the structure explicitly shows how the patient met the "OR" condition for BMI, providing a transparent and intuitive summary for clinical review.

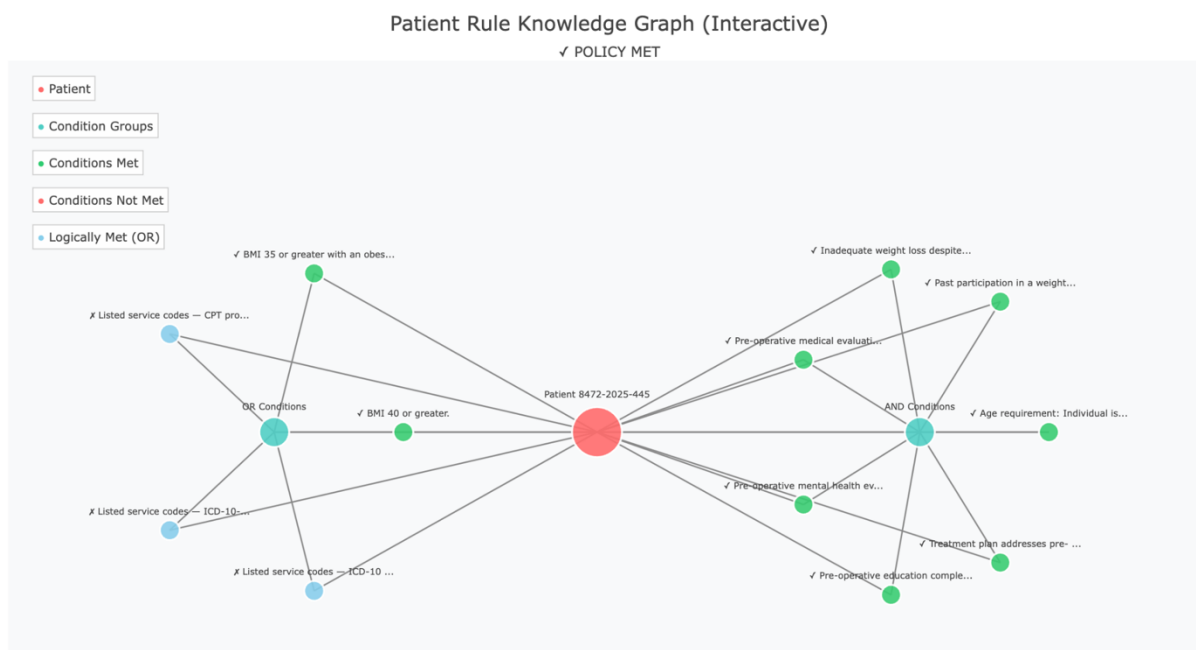


Figure 3. Patient Rule Knowledge Graph for Patient 8472202544 showing "Eligible" status

4.5. Compliance Evaluation Architecture

Condition-Level Evaluation. Each policy condition is evaluated by comparing the patient's clinical value with the policy threshold using data-type-aware logic. The engine determines whether each condition is satisfied, unmet, or logically satisfied through alternative OR paths. This three-state condition model enables accurate handling of complex nested policy logic.

Global Compliance Reasoning. Overall compliance is determined by verifying that all AND-linked conditions are logically satisfied and that at least one condition in each OR group is met. If these requirements hold, the patient is deemed compliant; otherwise, a non-compliance determination is generated with explicit identification of failing conditions.

Compliance Report and Integrated Knowledge Graph Generation. The system outputs a detailed JSON-formatted compliance report documenting patient identifiers, policy identifiers, individual condition evaluations, logical status, and the final coverage decision. Additionally, an integrated patient-policy knowledge graph is generated, in which condition nodes are color-coded based on satisfaction status. This visualization provides immediate interpretability, supports auditing, and enables rapid clinical and administrative review.

4.6. Validation

This section validates the quantitative accuracy of the Knowledge Graph-based Policy Automation System. Given the high stakes of healthcare decision-making and the limited availability of ground-truth annotated datasets, this evaluation is preliminary. It is intended to establish a technical baseline for the system's reasoning capabilities before the solution is scaled and clinically validated by the client.

The evaluation focused on two representative policies: Bariatric Surgery and Peripheral Nerve Stimulation (PNS). The specific results detailed below focus on the PNS policy to demonstrate the system's handling of complex neurostimulation criteria.

4.6.1. Methodology: Structured LLM-as-a-Judge

To ensure objective scoring despite limited data, we adopted a robust **LLM-as-a-Judge** methodology. We assessed the performance of the agentic extraction system across three phases: (1) Data Dictionary Extraction, (2) Policy Extraction, and (3) Patient Record Extraction.

To minimize bias, we conducted multiple independent evaluations per phase, varying both the model and the strategy:

- **Judge Models:** We utilized diverse Large Language Models (Claude 4.5 Sonnet and Gemini 3 Pro) to provide distinct reasoning perspectives.
- **Prompting Strategy:** We employed multiple prompt styles, including Zero-shot (direct assessment) and Chain-of-Thought (CoT), to test for consistency in evaluation.
- **Documentation of Results:** Detailed performance breakdowns by model and strategy are provided in **Appendix B**. Due to the extensive length of the raw prompt chains and JSON outputs, full transcripts of the interactions are provided in the separate **Technical Appendix** document.
- **Consensus Metrics:** For each phase, the judge evaluations were aggregated to produce the final consensus metrics presented in the tables below.

4.6.2. Phase 1: Data Dictionary Extraction

Objective: Evaluate how accurately the system identifies structured variable definitions (e.g., CPT codes, coverage booleans, duration thresholds) from the PNS policy text.

Metric	Score	Assessment
Precision	0.974	Near Perfect: The system rarely "hallucinated" fields. Almost every extracted variable (e.g., peripheral_nerve_stimulation_coverage) was supported by the text.
Recall	0.676	Moderate: The system missed some administrative nuances, such as specific provider specialty requirements (e.g., Board Certification in Pain Medicine) and place-of-service mandates.
F1-Score	0.793	Strong Baseline: A solid foundation for building the knowledge graph, though human review is recommended for administrative edge cases.

Table 1: Phase 1 Average Performance (PNS Policy)

4.6.3. Phase 2: Policy Condition Extraction

Objective: Evaluate the system's ability to convert complex natural language clauses (e.g., "50% pain reduction required") into computable logical rules.

Metric	Score	Assessment
Precision	0.969	High Reliability: The system accurately captured the logic for core rules, such as the 50% pain reduction threshold and exclusion of field stimulation.
Recall	0.894	High Completeness: The system successfully extracted the vast majority of eligibility criteria, only missing minor edge cases regarding specific non-covered diagnosis string matching.
F1-Score	0.928	Excellent: This is the highest-performing phase, indicating the system is highly effective at structuring clinical policy logic.

Table 2: Phase 2 Average Performance (PNS Policy)

4.6.4. Phase 3: Patient Condition Extraction

Objective: Evaluate the accuracy of extracting clinical facts (e.g., contraindications, prior failures) from unstructured patient medical records.

Metric	Score	Assessment
Precision	0.917	Safe Extraction: The system did not infer clinical facts that were not present in the text (e.g., it correctly identified the CPT code).

Recall	0.609	Conservative: The system missed some implicit concepts, such as calculating "chronic pain duration" (84 months) from a text stating "7 years duration," or inferring specific ICD codes not explicitly listed in the summary.
F1-Score	0.729	Good: The extraction is reliable for explicit facts but requires refinement for inferential reasoning (e.g., calculating dates or mapping synonyms).

Table 3: Phase 3 Average Performance (Patient Record L37360_001)

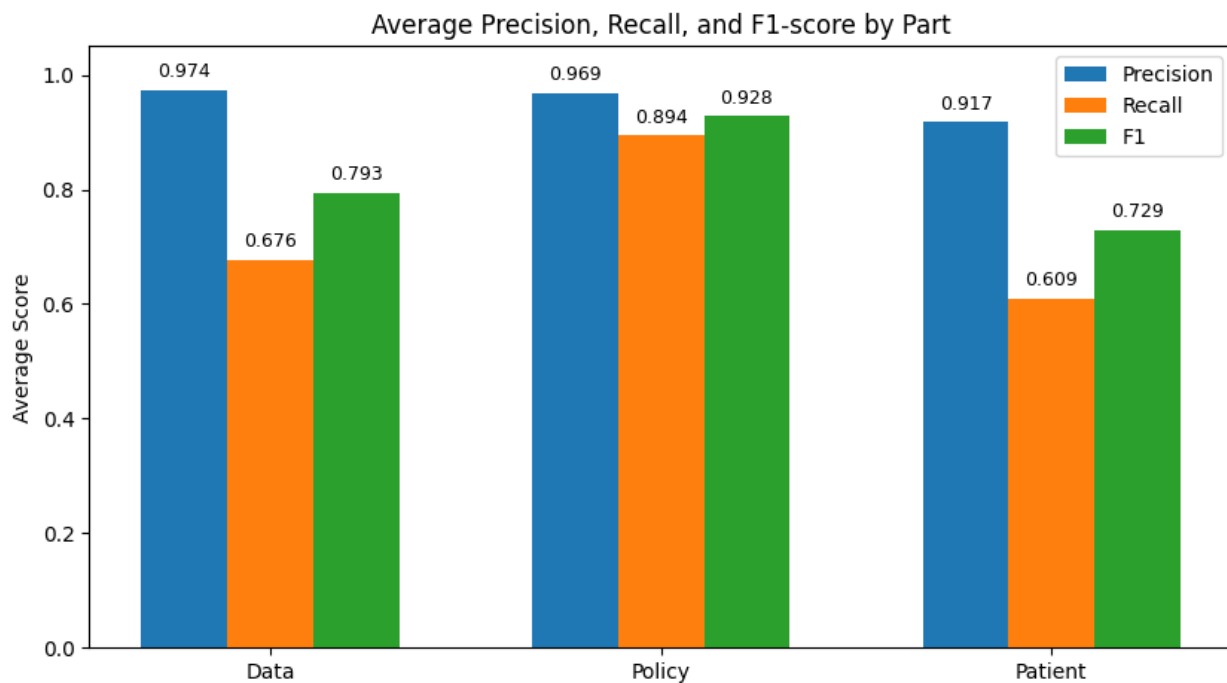


Figure 4: Summary of System Performance across Extraction Phases.

From **Figure 4**, it can be inferred that precision remains consistently high (>0.90) across all phases. This metric is critical for healthcare automation, as it ensures the system prioritizes safety by minimizing false positives (hallucinations).

4.6.5. Join Technical Validation

Beyond the automated metrics, a rigorous qualitative review was conducted to validate the system's "reasoning chain." This technical validation was performed jointly by the CMU Capstone Team (Lohitaksha Ananthula and Simon) and the Zyter Technical Team.

Both parties reviewed the extracted Knowledge Graphs and the intermediate JSON outputs for the Bariatric and PNS policies. The joint review confirmed:

1. **Structural Integrity:** The Knowledge Graphs accurately reflected the hierarchical dependencies of the source PDFs.
2. **Output Agreement:** Both teams agreed that the pipeline's output was effective with some weaknesses and is ready for integration into the broader Zyter platform.

4.6.6. Future Clinical Validation

The core technical pipeline has been established and documented in the project repository, providing a functional foundation for further development. Future clinical validation will now be driven by the Zyter team as they integrate and adapt this codebase for in-house piloting. This next phase will focus on testing the pipeline against a larger, multi-patient dataset to determine if the high precision observed in our initial benchmarks holds true at scale. Additionally, the system's "Medical Necessity" outputs will require rigorous review by licensed clinicians to ensure patient safety and regulatory compliance before live deployment.

4.7 Limitations and Future Work

The system offers several opportunities for further development as it moves toward production-ready deployment. While the current prototype demonstrates the viability of the architecture, several constraints remain that define the roadmap for future work.

Current Limitations

- **Unstable Extraction:** LLM-based extraction may fail on rare edge cases and misinterpret deeply nested or complex logical structures in policy text.
- **Lack of Oversight:** The current multi-agent pipeline lacks a Human-in-the-Loop (HITL) validation mechanism for low-confidence or high-risk decisions.
- **Data Verification:** Patient records still rely heavily on OCR-based and manually generated JSON data without systematic clinical value verification.
- **Limited Interoperability:** Medical code coverage is currently limited to CPT and ICD-10, limiting the system's ability to map to broader clinical ontologies.

Future Roadmap

- **Advanced OCR:** Future OCR improvements will target low-quality scans, complex multi-column layouts, tables, and non-standard document formatting using layout-aware and multimodal approaches.
- **Dedicated Validation Agent:** A specialized validation agent will be introduced to cross-check extracted data fields, policy logic, and generated SQL queries.
- **Human-in-the-Loop Integration:** A review interface will be integrated to allow clinicians to audit and edit intermediate agent outputs directly within the Streamlit application, ensuring safe deployment in high-risk scenarios.
- **Gold-Standard Benchmarking:** A manually annotated policy dataset will be constructed to enable large-scale benchmarking and evaluation.
- **Expanded Ontology:** Medical code coverage will be expanded via UMLS integration to support broader clinical interoperability.

5. Voice Agent (Post-Discharge Orchestration System)

5.1. System Architecture and Technology Stack

5.1.1. Operational Challenges in Post-Discharge Care

Before detailing the technical architecture, it is critical to understand the specific clinical and operational challenges this system addresses. Managing post-discharge patients is a complex

safety challenge, as nearly 20% of Medicare patients are rehospitalized within 30 days of discharge, often due to preventable complications (Jencks et al., 2009). The system was designed to address three specific barriers to continuity of care where conversational agents have shown promise for improving engagement (Laranjo et al., 2018):

- **The Medication Adherence Challenge:** Providing medication advice via AI is inherently risky. A patient asking, "What happens if I miss a dose?" requires a standard answer, but a patient stating, "I accidentally took two doses," creates a potential toxicity event. Conversational agents must be rigorously validated to distinguish educational queries from active overdose risks.
- **The Appointment Triage Challenge:** Scheduling in healthcare is non-deterministic and depends on urgency. Existing symptom checkers often struggle with consistent triage accuracy, with audit studies showing significant variability in diagnostic and triage performance (Semigran et al., 2015). It is dangerous to automate scheduling without a clinical safety layer; a patient requesting a visit for "chest pain" requires immediate emergency redirection, not a calendar slot.
- **The Latency Challenge:** In the current client workflow, patients with concerning symptoms often leave voicemails for nurses, resulting in callback windows of several hours. This latency can be dangerous for conditions where minutes matter.

5.1.2. Modular Multi-Agent Architecture

The Voice Agent is built on a modular multi-agent architecture using the LangGraph framework. This design ensures coordination between flexible LLM reasoning and auditable, deterministic control flow.

- **Orchestration Agent:** Performs hybrid intent detection, using an LLM classifier backed by rule-based validation. This agent routes the request to the correct specialist while preserving context.
- **Specialized Agents:** Four domain specific agents (Appointment, Follow-up, Medication, Caregiver) handle the request. Each is an independent logic component with its own validation rules and structured outputs.

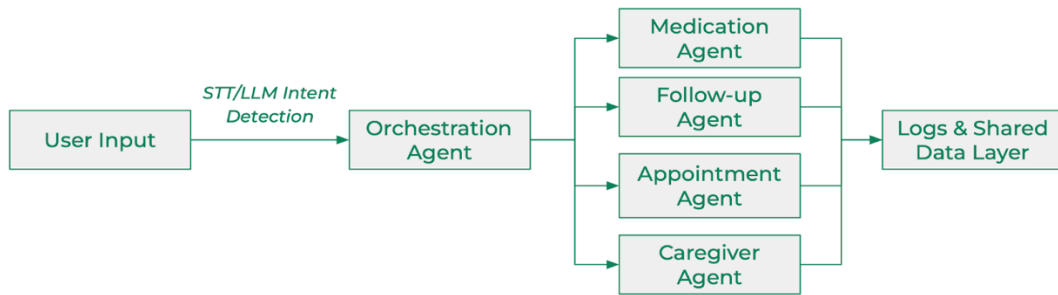


Figure 5. Overall Multi-Agent System Architecture

5.1.3. Key Technical Components

Component	Role	Description
State Management	Central context and short-term memory.	VoiceAgentState ensures unidirectional data flow and consistency across all agents.
LLM Provider Abstraction	Unified interface for chat completion and NLU.	llm_provider.py supports OpenAI, Anthropic, and Google Gemini, allowing for model switching to manage cost and risk.
Data Service Layer	Abstracted access to all patient data.	Uses local CSV files to simulate patient data structures, making the system ready for future FHIR/EHR integration .
Safety Design	Enforcement of minimum safety guarantees.	Dual-layer safety checks: LLM inference is complemented by deterministic rules to ensure predictable behavior in critical paths.

Table 4: Key Technical Components

5.2. Core Agent Logic and Implementation

The system's safety and clinical utility are derived from the structured logic within its specialized agents.

- **Orchestration and Intent Detection:** This agent employs a production-grade routing strategy to ensure requests reach the correct specialist. It uses an LLM classifier for naturalistic queries, backed by a rule-based fallback that applies keywords and priority ordering. Critical entities like patient ID are validated before passing control downstream.
- **Appointment Agent:** This agent integrates scheduling with a three-tier triage framework. The team designed pattern-based detectors for high-risk symptoms like chest pain. Crucially, a RED-tier detection immediately overrides the scheduling request and directs the user to emergency guidance.
- **Follow-up Agent:** Responsible for accurate symptom logging, this agent prompts the LLM to extract data into a structured JSON schema (severity, location, timing). To ensure robustness during potential API outages, it includes a dictionary-based fallback extractor. It aligns strictly with the Appointment Agent's triage rules for consistency.
- **Medication Agent:** Development focused on conservativeness. The agent queries a structured drug knowledge base (side effects, contraindications) and normalizes drug names. It implements a RED/ORANGE/GREEN risk classification; high-risk interactions or misuse scenarios always trigger safety warnings rather than speculative guidance.
- **Caregiver Agent:** Enforces strict privacy logic. It verifies caregiver relationships and documented patient consent before aggregating log data from other agents. It then uses an LLM to generate a clear, compliant weekly narrative summary.

5.3. Methodology and Development Approach

5.3.1. Clinical Workflow Analysis and Threat Modeling

To mitigate the risks identified in Section 5.1—specifically the variability of symptom triage and the high stakes of medication errors—the team conducted a comprehensive analysis of real-world clinical workflows. This was followed by a rigorous threat modeling exercise to identify potential failure modes, such as LLM hallucinations or ambiguous patient inputs.

This proactive risk management approach aligns with the World Health Organization’s guidance on AI governance, which emphasizes human oversight and safety in clinical AI deployment (World Health Organization, 2021; World Health Organization, 2024).

Furthermore, the design of our safety guardrails and fallback mechanisms adheres to the NIST AI Risk Management Framework, specifically regarding the management of system reliability and safety in high-consequence environments (National Institute of Standards and Technology, 2023). By mapping potential failure points (e.g., a patient minimizing severe chest pain) directly to agent behaviors, we ensured the system defaults to safety in uncertainty.

5.3.2. Prompt Engineering Strategy

To ensure reliability, the system employs deterministic prompt engineering. Temperature parameters for classification tasks are set to zero to minimize variability. Prompts are structured to explicitly request JSON-formatted outputs with strict schema definitions. This strategy decouples the reasoning capability of the LLM from the formatting requirements of the system, ensuring that downstream code can reliably parse agent outputs.

5.3.3. Design Principles: Dual-Layer Safety

Each feature was implemented with dual-layer safety checks. Language model inference provides flexibility in natural language understanding, while deterministic rules enforce minimum safety guarantees. This ensures the system behaves reliably even when LLM output is incomplete or unavailable.

5.3.4. Iterative Validation Strategy

The development process utilized an automated validation dataset generator to execute test cases through the actual system workflow. This allowed for continuous integration testing where real responses were captured and compared against clinical ground truth. This iterative cycle enabled the team to rapidly identify and fix pattern matching limitations, such as the initial failure to detect chest tightness effectively.

5.3.5. Logging and Auditability

A consistent logging approach was implemented throughout development. All agent inputs, outputs, classifications, and fallback events are captured in structured JSON formats. This methodology aligns the system with auditability expectations for real-world healthcare AI deployments.

5.4. Validation and Data Analysis

The system was rigorously evaluated on a curated dataset to confirm its accuracy, safety, and performance.

5.4.1. Data Sources and Limitations

The evaluation of the post-discharge voice system was conducted on a small, curated dataset created to support core logic flows like symptom triage, medication reasoning, and appointment workflows. Early development relied on a mix of MIMIC-inspired templates and LLM-generated synthetic examples, while the current version uses 3–10 structured patient records tailored to exercise the intended use cases.

This dataset provides clarity and control for early prototyping but introduces several limitations. First, its small size restricts the system’s exposure to real-world clinical variability, including diverse symptom expressions, medication regimens, and follow-up scenarios. Second, curated and synthetic records do not fully capture the inconsistency and fragmentation seen in production EHR systems. As a result, system performance may differ when expanded to larger or more heterogeneous patient populations. These data constraints highlight the need for broader and more realistic datasets to strengthen robustness and support deployment in operational environments.

5.4.2. Validation Outcomes

The system was evaluated on its ability to route user inputs, classify medication-related intents, assign medication risk levels, and triage follow-up severity. Across these tasks, the system demonstrated strong performance on a curated evaluation set designed to reflect common post-discharge scenarios.

In addition to classification outcomes, the system satisfied real-time interaction requirements, with an average end-to-end latency of 1.47 seconds. Because the evaluation set reflects controlled conditions rather than real-world variability, these results should be interpreted as upper-bound performance.

Evaluation Metric	Goal/Standard	Actual Result	Implication
Policy Compliance	100%	100%	All business rules were executed without failure.
Medication Risk Identification	$\geq 90\%$	100%	The system reliably detected and flagged high-risk drug scenarios.
Intent Routing Accuracy	$\geq 95\%$	100%	Ensures requests consistently reach the correct specialized agent.
Average Latency (End-to-End)	< 2 seconds	1.47 seconds	Meets performance requirements for real-time conversational interaction.

Table 5: Key Performance Metrics

5.4.3. Confusion Matrix Analysis

Figure 6 presents a consolidated confusion matrix summarizing classification outcomes across all components. Correct predictions are shown in green and errors in red, allowing the overall error

patterns to be interpreted at a glance. Reformatted versions of each individual matrix are included below the figure for clarity.



Figure 6. Consolidated confusion matrices for all classification components

5.5. Future Work

The system offers several opportunities for further development as it moves toward production-ready deployment.

- **Zyter Platform Integration:** The multi-agent orchestration layer can be integrated directly into Zyter’s existing platform, allowing policy reasoning, symptom triage, caregiver summaries, and appointment management to operate within a unified modular infrastructure.
- **Dataset Expansion:** Expanding the dataset with more realistic mock EHR records and a broader range of discharge scenarios would improve the robustness of triage logic and intent interpretation.
- **Strengthen Orchestration Rules:** Strengthening the orchestration rules would help meet production-level completeness, safety expectations, and additional operational requirements.

- **Multi-Turn Dialogue Memory:** Another direction is the introduction of multi-turn dialogue memory, enabling the system to maintain context across longer interactions and provide more continuous support.
- **Monitoring Dashboards:** Developing monitoring and analytics dashboards would provide caregivers, clinical staff, and supervisors with clearer visibility into patient trends, agent performance, and system reliability.

6. Lessons Learned

- **Deterministic Control is Essential for Healthcare AI-** Early iterations of the voice agent relied too heavily on "pure" LLM prompting for triage, which led to inconsistent safety classifications. Migrating to LangGraph allowed us to enforce strict boundaries. We learned that while LLMs are excellent for natural language understanding (NLU), critical clinical decisions (Triage Red/Green) must be guarded by deterministic code and structured policies.
- **In the Knowledge Graph project-** Extracting clean text from medical policies was straightforward for body text but extremely difficult for nested tables and multi-column layouts. We learned that a hybrid approach combining PyMuPDF for native digital PDFs and Tesseract for scanned images was necessary. Furthermore, passing the OCR output through a 'Data Dictionary Agent' first significantly improved the accuracy of downstream logic extraction compared to feeding raw text directly to the logic generator.
- **Latency vs. Accuracy Trade-off-** For the voice agent, we initially struggled with high latency using the largest available models. We optimized this by implementing a "Router" node that uses a faster, smaller model to detect intent, only calling the larger, more reasoning-capable models for complex medical questions. This brought average response times down to under 1.5 seconds.
- **Multi-Agent Specialization Wins-** Attempting to build a single "Super Agent" to handle appointments, medications, and symptoms resulted in context leakage and hallucination. Decomposing the system into specialized agents (e.g., a Medication Agent that only knows drug interactions) drastically improved reliability and made debugging specific errors much easier.

7. Conclusion

The Zyter-CMU Capstone project has explored the potential for integrating Generative AI into high-stakes healthcare workflows. Our findings suggest that when wrapped in a robust, modular architecture, these technologies can effectively address critical gaps in both administrative efficiency and patient continuity of care.

The Knowledge Graph system indicates that even complex, non-standardized payer policies can be digitized into computable logic. This suggests a scalable pathway to reduce the administrative burden of manual eligibility reviews while maintaining the auditability required for compliance. Similarly, the Voice Orchestration system demonstrates the feasibility of using AI to augment clinical staff. By implementing strict safety guardrails, the system showed that conversational agents can handle routine triage and medication inquiries with a high degree of reliability, potentially acting as a force multiplier for care teams.

Together, these prototypes offer a promising framework for the future of connected health. They illustrate that a hybrid approach, combining the flexibility of LLMs with the safety of deterministic code, can support the shift from reactive, manual operations to proactive, data-driven automation.

8. Recommendations

- **Human-in-the-Loop Validation Interface:** While the extraction accuracy is high, it is not perfect. We recommend building a "Clinician Review Dashboard" where the generated Knowledge Graph and SQL queries are presented to a human expert for approval before being deployed into production.
- **Expand OCR Capabilities:** The current OCR pipeline struggles with low-quality faxed records, which are common in healthcare. Future work should integrate multi-modal vision models (like GPT-4o or Gemini 1.5 Pro) to "read" document snapshots directly rather than relying solely on text-layer extraction.
- **Integration with EHR APIs:** Currently, the systems operate on local CSV/JSON datasets. To be production-ready, the system must support HL7/FHIR standards to pull

real-time patient data (e.g., current medications, recent vitals) from Electronic Health Records (EHR) systems like Epic or Cerner.

- **Dataset Diversification:** The current validation relied on 122 curated test cases. To ensure robustness against different accents, dialects, and complex comorbidities, the voice system should be tested against a larger, more diverse dataset of de-identified patient calls.

References

- Boxwala, A. A., Rocha, B. H., Maviglia, S., Kashyap, V., Meltzer, S., Kim, J., & Ohno-Machado, L. (2011). A multi-layered framework for disseminating knowledge for computer-based decision support. *Journal of the American Medical Informatics Association*, 18(suppl_1), i132–i139. <https://doi.org/10.1136/amiajnl-2011-000316>
- Centers for Disease Control and Prevention. (2024, September). Outpatient follow-up visits to reduce 30-day all-cause readmissions: A systematic review and meta-analysis. *Preventing Chronic Disease*. https://www.cdc.gov/pcd/issues/2024/24_0138.htm
- Change Healthcare. (2022). The Change Healthcare 2022 revenue cycle denials index. <https://blr.postclickmarketing.com/Global/FileLib/HLM/2022-revenue-cycle-denials-index.pdf>
- Davenport, T., & Kalakota, R. (2019). The potential for artificial intelligence in healthcare. *Future Healthcare Journal*, 6(2), 94–98. <https://doi.org/10.7861/futurehosp.6-2-94>
- Jencks, S. F., Williams, M. V., & Coleman, E. A. (2009). Rehospitalizations among patients in the Medicare fee-for-service program. *The New England Journal of Medicine*, 360(14), 1418–1428. <https://doi.org/10.1056/NEJMsa0803563>
- Johnson, A. E. W., Pollard, T. J., Shen, L., Lehman, L. W. H., Feng, M., Ghassemi, M., ... & Mark, R. G. (2016). MIMIC-III, a freely accessible critical care database. *Scientific Data*, 3, 160035. <https://doi.org/10.1038/sdata.2016.35>
- Kawamoto, K., Houlihan, C. A., Balas, E. A., & Lobach, D. F. (2005). Improving clinical practice using clinical decision support systems: A systematic review of trials to identify features critical to success. *BMJ*, 330(7494), 765. <https://doi.org/10.1136/bmj.38398.500764.8F>
- Laranjo, L., Dunn, A. G., Tong, H. L., Kocaballi, A. B., Chen, J., Bashir, R., ... & Lau, A. Y. (2018). Conversational agents in healthcare: A systematic review. *Journal of the American Medical Informatics Association*, 25(9), 1248–1258. <https://doi.org/10.1093/jamia/ocy072>

National Institute of Standards and Technology. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0) (NIST AI 100-1). U.S. Department of Commerce.

<https://doi.org/10.6028/NIST.AI.100-1>

Reducing readmissions in the safety net through AI and automation. (2025, March). The American Journal of Managed Care. <https://www.ajmc.com/view/reducing-readmissions-in-the-safety-net-through-ai-and-automation>

Rotmensch, M., Halpern, Y., Tlimat, A., Horng, S., & Sontag, D. (2017). Learning a health knowledge graph from electronic medical records. Scientific Reports, 7(1), 5994.

<https://doi.org/10.1038/s41598-017-05778-z>

Semigran, H. L., Linder, J. A., Gidengil, C., & Mehrotra, A. (2015). Evaluation of symptom checkers for self diagnosis and triage: Audit study. BMJ, 351, h3480.

<https://www.bmj.com/content/351/bmj.h3480>

Topol, E. J. (2019). High-performance medicine: The convergence of human and artificial intelligence. Nature Medicine, 25(1), 44–56. <https://doi.org/10.1038/s41591-018-0300-7>

World Health Organization. (2021). Ethics and governance of artificial intelligence for health. <https://www.who.int/publications/i/item/9789240029200>

World Health Organization. (2024). Ethics and governance of artificial intelligence for health: Guidance on large multi-modal models. <https://www.who.int/publications/i/item/9789240084759>

Appendix

Appendix A: Technical Implementation and Deployment

This section summarizes the technical implementation, project structure, and deployment instructions for both subsystems.

Central Repository & Overview

The complete source code, unified README, and documentation for both the Knowledge Graph and Voice Agent subsystems are available at:

https://github.com/sijiasiga/Capstone_KG_VoiceAgents/tree/main

A. Medical Policy Knowledge Graph System

Technical Setup

This component utilizes a multi-agent pipeline orchestrated via Python to process PDF documents. It leverages Google Gemini 1.5 Pro/Flash for agent reasoning and Streamlit for the user interface.

- **Sub-Repository:** https://github.com/sijiasiga/Capstone_KG_VoiceAgents/tree/main/KG
- **Key Dependencies:** PyMuPDF (OCR), Streamlit (UI), Google Generative AI SDK (LLM), NetworkX (Graph plotting).

Project Structure

The system is organized around a three-agent pipeline located in the KG/ directory:

- **process_policy.py:** The master orchestration script that chains the DataField, Policy, and SQL agents.
- **OCR/:** Contains modules for text extraction from complex PDF layouts.
- **patient_data/:** Stores synthetic patient records (e.g., patient_8472202544) and their generated JSON profiles.

- **Policies_test/:** Stores the output artifacts, including the extracted Data Dictionaries, Policy Logic JSONs, and generated SQL queries.

Architecture and Data Flow

1. **Input:** Raw PDF Policy (e.g., Bariatric Surgery CGSURG83).
2. **Agent 1 (DataField):** Extracts variable definitions into a JSON Data Dictionary.
3. **Agent 2 (Policy):** Extracts logical conditions (AND/OR rules).
4. **Agent 3 (SQL):** Converts conditions into executable WHERE clauses.
5. **Output:** A visual Knowledge Graph (policy_rule_kg.png) and a Compliance Report JSON.

Deployment Instructions

To run the Policy Extraction and Validation interface:

```
cd KG

# Ensure api.json contains your Gemini key

pip install -r requirements.txt

streamlit run streamlit_final.py
```

B. VoiceAgents - Healthcare Voice Triage System

Technical Setup

The voice system is built on LangGraph to ensure deterministic control flow over LLM outputs. It supports a multi-provider fallback strategy (Anthropic, Google, OpenAI) to ensure high availability.

- **Sub-Repository:**
https://github.com/sijjasiga/Capstone_KG_VoiceAgents/tree/main/VoiceAgents

- **Key Dependencies:** LangGraph (Orchestration), OpenAI/Anthropic SDKs, Streamlit (Demo UI).

Project Structure

The core logic resides in `VoiceAgents_langgraph/`:

- **workflow.py:** Defines the graph topology (nodes and edges).
- **state.py:** Defines the `VoiceAgentState` schema passed between nodes.
- **nodes/:** Contains specific domain logic (`appointment.py`, `medication.py`, `routing.py`).
- **policy/:** Stores JSON configuration files for clinical safety boundaries.
- **logs/:** Centralized storage for interaction logs (`appointment_agent_log.jsonl`, `symptom_logs.csv`).

Triage Configuration

The system uses a strict 3-tier safety configuration defined in `policy/`:

- **RED:** Emergency (e.g., Chest pain) $\rightarrow$ Immediate redirect.
- **ORANGE:** Concerning (e.g., Moderate fever) $\rightarrow$ Nurse callback.
- **GREEN:** Routine (e.g., Scheduling) $\rightarrow$ Automated handling.

Deployment Instructions

To run the CLI version with Voice (TTS) enabled:

```
cd VoiceAgents_langgraph
# Configure .env with API keys
pip install -r requirements.txt
python main.py
# Inside the CLI, type ':voice on' to enable Text-to-Speech
```

Appendix B: Detailed Validation Metrics by Model & Strategy

This appendix provides the granular performance breakdown for each phase of the extraction pipeline. The metrics below compare the performance of Gemini 3 Pro and Claude 4.5 Sonnet using both Zero-Shot and Chain-of-Thought (CoT) prompting strategies.

(**Note:** For the full transcripts of the raw prompts and the corresponding JSON outputs generated by the models, please refer to the separate **Technical Appendix** document.)

Phase 1: Data Dictionary Extraction

Objective: Extract structured variable definitions from policy text.

Model	Prompt Strategy	Precision	Recall	F1-Score	True Pos	False Pos	False Neg
Gemini 3 Pro	Zero-Shot	1.000	0.826	0.905	19	0	4
Claude 4.5	Zero-Shot	0.950	0.600	0.730	18	1	12
Gemini 3 Pro	Chain-of-Thought	0.947	0.720	0.818	18	1	7
Claude 4.5	Chain-of-Thought	1.000	0.559	0.717	19	0	15

Phase 2: Policy Condition Extraction

Objective: Convert policy clauses into computable logical rules.

Model	Prompt Strategy	Precision	Recall	F1-Score	True Pos	False Pos	False Neg
Gemini 3 Pro	Zero-Shot	1.000	0.914	0.955	32	0	3
Claude 4.5	Zero-Shot	0.938	0.938	0.938	30	2	2
Gemini 3 Pro	Chain-of-Thought	0.969	0.969	0.969	31	1	1
Claude 4.5	Chain-of-Thought	0.969	0.756	0.849	31	1	10

Phase 3: Patient Record Extraction

Objective: Extract clinical facts from patient medical records.

Model	Prompt Strategy	Precision	Recall	F1-Score	True Pos	False Pos	False Neg
Gemini 3 Pro	Zero-Shot	1.000	0.600	0.750	6	0	4
Claude 4.5	Zero-Shot	0.670	0.570	0.620	4	2	3
Gemini 3 Pro	Chain-of-Thought	1.000	0.546	0.706	6	0	5
Claude 4.5	Chain-of-Thought	1.000	0.722	0.839	13	0	5